

SESSION - 13

Components Hands On



Table of Contents

Table of Contents **1**

Components Hands On ***2***

Components ***2***

Structure of a Component ***3***

Root Node **3**

Vital Properties **3**

Vital Child Nodes **3**

Structure Components and Content Components ***3***

HTL Business Logic ***4***

Sling Models **4**

Server-Side JavaScript **4**

Exercise 1: Create a header component **5**

Exercise 2: Add JavaScript business logic to the header component **9**

Exercise 3: Add a Sling Model business logic to the header component **10**

Configuring the Edit Dialog ***14***

Exercise 4: Create an edit dialog for the component **14**

Exercise 5: Add an editconfig node to the component **20**

Exercise 6: Add a component client library to the component **21**

Design Dialogs ***23***

Exercise 7: Create a design dialog for a component **24**

Components Hands On

Introduction

In addition to the wide range of default components available in Adobe Experience Manager (AEM), you can develop new components that offer flexible, feature-rich authoring functionality. To develop components in AEM, you must understand the structure of the components and its configuration, component hierarchy and inheritance, component edit behaviour, HTML Template Language (HTL) business logics, and design configuration through content policies.

Objectives

After completing this module, you will be able to:

- Describe the structure of a component
- Describe structure components and content components
- Explain the HTL business logic
- Create a header component
- Add JavaScript business logic to a header component
- Add Sling Model business logic to a header component
- Create an edit dialog for the component
- Add an **editconfig** node to the component
- Add a component client library to the component
- Explain AEM design dialogs
- Create a design dialog for a component

Components

In AEM, components:

- Are the structural elements that constitute the content of the pages being authored.
- Are resource types.
- Contain a collection of scripts that implement a specific functionality to present the content on the website.
- Are modular and reusable units.
- Are developed as self-contained units within one folder of the repository.
- Have no hidden configuration files.
- Can contain other components.
- Work anywhere within any AEM system.
- Have a standardized user interface.

Structure of a Component

The component structure includes:

- Root node
- Vital properties
- Vital child nodes

Root Node

It is the hierarchy node of the component and represented as node `<mycomponent>` (type `cq:Component`).

Vital Properties

The vital properties of a component are:

- jcr:title: Is the component title. For example, this property is used as a label when the component is listed in the components browser.
- jcr:description: Is the description for the component. You can use this property as a mouse-over hint in the components browser.
- cq:icon: Is the string property pointing to a standard icon in the Coral UI library to display in the component browser.

Vital Child Nodes

The vital child nodes of a component are:

- cq:editConfig (cq:EditConfig): Defines the edit properties of the component and enables the component to appear in the components browser
- cq:childEditConfig (cq:EditConfig): Controls the author UI aspects for child components that do not define their own cq:editConfig
- cq:dialog (nt:unstructured): Is the dialog for this component and defines the interface allowing the user to configure the component and/or edit content.
- cq:design_dialog (nt:unstructured): Helps edit the design of a component

Structure Components and Content Components

The content of a page is distributed into layouts and page-specific categories. You can design the page layout with a set of components called structure components. The content that is unique to the page is added through content components.
Authors should follow the following guidelines when working with structure and content components:

- Template authors can add and remove the structure components to the template.
- Authors cannot remove structure components from the page.
- Authors can edit and configure structure components of the page.

- Authors can add and remove content components from the page.

HTL Business Logic

HTL is HTML5 that helps develop and design business logic in HTL completely. HTL helps you add complex business logic to the code through the following options:

1. **Sling Models**

2. **Server-Side JavaScript**

Each method exposes global objects that can be used in the business logic. You can call the getters [get() method] in HTL to expose the output of the above methods. The following HTL statement is used for invoking the business logic:

*<div data-sly.use.myObj="com.my.package.MyClass">${myObj.myGetter} *

Sling Models

Sling Models is the preferred method for implementing the business logic of a component. Sling Models are Java classes used to develop components. Sling Models support both HTML components and a headless Content Management System (CMS).
Sling Models are Java classes that can be mapped to Sling resources. To support a headless CMS, you can also use the Sling Model Exporter to export the resource (the node representing the component) as JavaScript Object Notation (JSON). Apache Sling provides a Jackson JSON exporter to cover the most common case of exporting Sling Models as JSON objects. The JSON objects are used by programmatic web consumers such as other Web services and JavaScript applications.

Server-Side JavaScript

Server-side JavaScript is made available through the Use-API. This API is from the HTL specification and helps developers write quick business logic for their components. Generally, business logic is built in Java. When the component needs some view-specific changes from what the Java API provides, it is convenient to use server-side executed JavaScript to do that.

```
use(function () {
var curTitle = currentPage.getTitle();
return {
link: "#my link's safe",
title: curTitle,
text: "my text's safe"
};
});
```

Exercise 1: Create a header component

In this exercise, you will create a header component and add it to the template.

1. In your AEM author instance, click Adobe Experience Manager from the header bar. The Tools console opens.
2. Click CRXDE Lite. The CRXDE Lite page opens.
3. Navigate to the /apps/training/components/structure folder.
4. Right-click the structure folder and click **Create** > **Create Component**. The Create Component dialog box opens.
5. Enter the following details in the dialog box:

   a. **Label: header**

   b. **Title: We.Train Header**

   c. **Group: We.Train.Structure**
6. Click Next. The Advanced Component Settings are displayed. Leave these settings as is
7. Click OK. The component is created.
8. Click Save All from the actions bar.
9. Expand the header component and double-click header.jsp. The editor opens on the right.
10. Delete the existing content and click Save All.
11. Right-click header.jsp, select Rename, and change the file name to header.html. Click Save All from the actions bar.
12. Click Save All from the actions bar.
13. Double-click the header.html to open it in the editor on the right for editing.
14. Navigate to the **basic_header.html** file located in the exercise folder provided to you, copy the code, and paste the code in **header.html.**
15. Click **Save All** to save changes.
16. Review the code you added. Notice how the code snippet of header.html has a simple navigation that lists the child pages of the current page. If there are no child pages, Nav1, Nav2, and Nav3 are displayed. The header.html also has the Language Navigation and Search proxy components coded in. You can configure these.

Before you add the header component to the template, you must add a cq:dialog node to the header component.

17. Right-click the header component and click Create > Create Node. The Create Node dialog box opens.
18. Provide the following details:

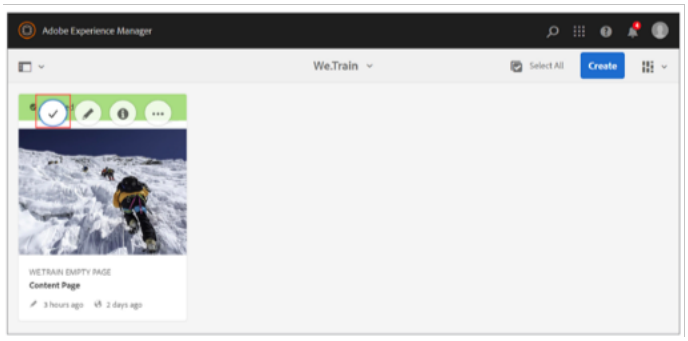
    a. Name: cq:dialog

b. Type: nt:unstructured

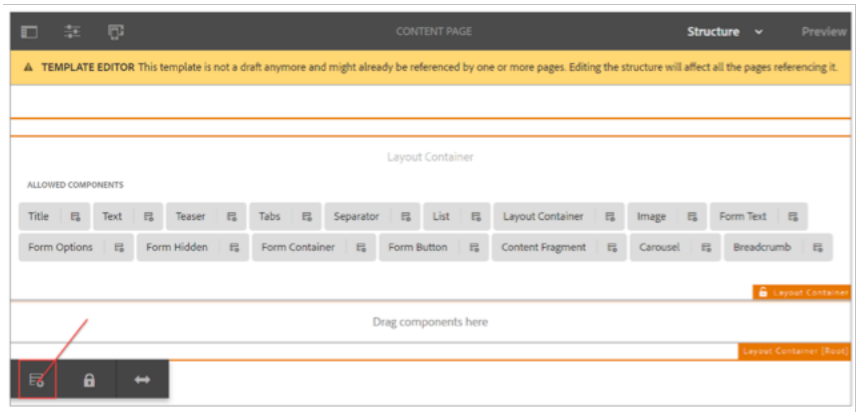
19. Click OK. The node is created.
20. Click Save All.
21. Click CRXDE Lite from the header bar to navigate back to AEM.

To add the header component to the template:

22. Click Tools > Templates. The Templates console opens.
23. Click the We.Train folder, and select the Content Page template by clicking the Select icon, as shown:



24. Click Edit (e) from the actions bar. The Template Editor opens.
25. Select the Layout Container [Root] and click the Policy icon from the component toolbar, as shown. The Layout Container policy wizard opens.

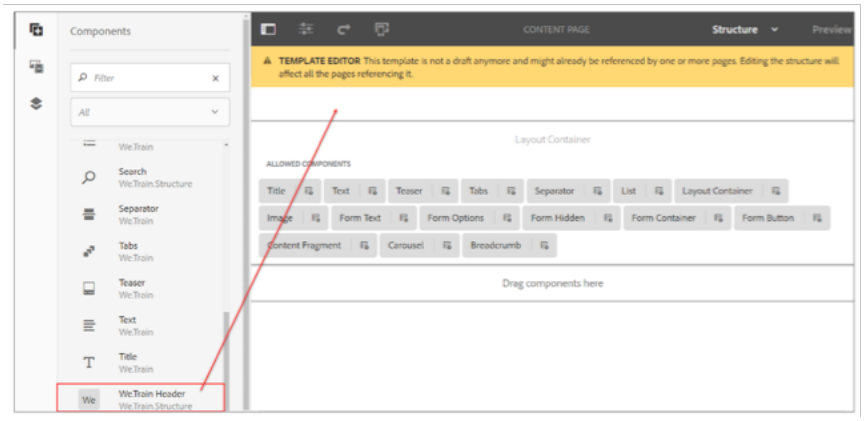


26. In the Properties section, on the Allowed Components tab, scroll down the components list, and select the We.Train.Structure checkbox. This group contains the header component.
27. Click Done (checkmark in the upper right) to save the changes.

Note: When adding the header component to the Content Page template, you did not create a new policy. Instead, you used the existing **We.Train Structure** policy. The next template that uses the

We.Train.Structure policy will automatically have access to the structure components (header) that you have built.

28. Click the Toggle Side Panel icon, if the left panel is not visible, and click the Components icon. The Components browser opens.
29. Drag the We.Train Header component from the panel and add it above the Layout Container, as shown. The Header Component is added to the template.



After adding the header component, you must set policies for the two proxy components.

30. Select the Language Navigation component and click the Policy icon from the component toolbar.
31. In the Policy section, type **We.Train LangNav** Content Policy in the Policy Title field.
32. In the **Properties** section, type **/content/we-train** in the Navigation Root field.
33. Click Done (checkmark) to save the changes.
34. Select the Search component and click the Policy icon.

Note: You could have configured the language navigation and search components by using a cq:template node too—similar to the carousel and tab proxy components. For example, review /apps/weretail/components/structure/header in CRXDE Lite.

35. Click http://localhost:4502/editor.html/content/we-train/en.html. The English page of the We.Train site with the header component (and its child pages) opens, as shown:

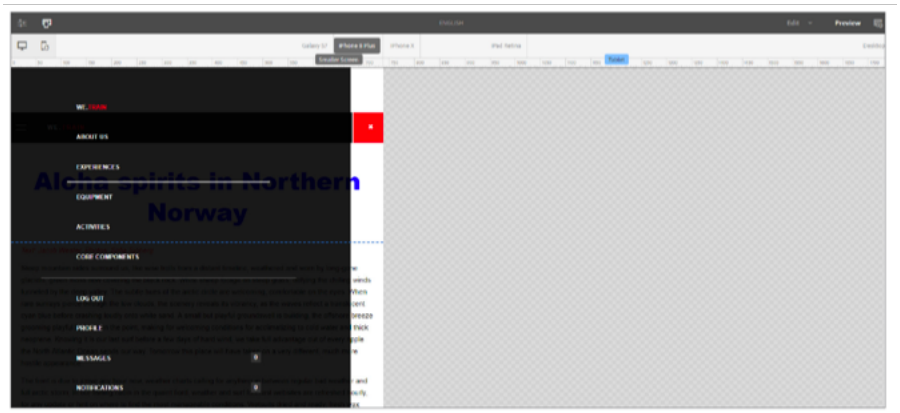


You created the component, added it to the template, and verified it on the page. Now, you can begin a deeper development with the header.html script.

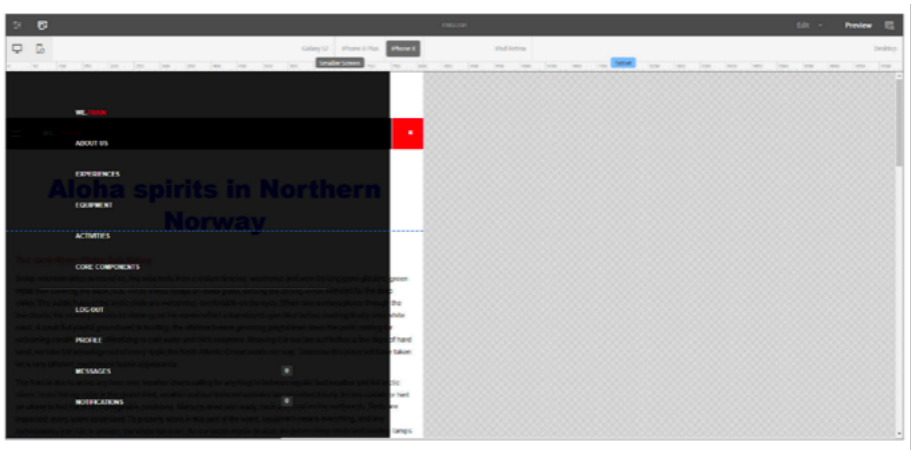
36. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
37. Navigate to the **/apps/training/components/structure/header** node.
38. Double-click the **header.html** to open the script for editing.

39. Update the header.html script by replacing the existing code with new code from **basic_full_header.html** exercise file provided to you.
40. Click Save All.
41. Review the code you added. Notice in the following code snippet of header.html, there is an account navigation bar and a responsive site navigation bar. Most anchors are href='#' that help mock the page for the business logic. Notice the **${header.rootPage.path}** and **${!header.anonymous}** variables from the business logic. As the header variable is not defined, it is not evaluated.
42. Navigate to the English page of the **We.Train** site and refresh the page.
43. Click Preview, and in the Preview mode, click the Emulator icon.
44. Click Desktop, iPhone 8 Plus, and iPhone X at the top, and notice that the header is responsive.

    iPhone 8 Plus



    iPhone X



You have now created a new structure component, added it to the editable template design, and extended it with a responsive design.

Exercise 2: Add JavaScript business logic to the header component

In this exercise, you will use JavaScript to add business logic to the header component. Using JavaScript is a quick way to add simple business logic to a component that is not needed for Content Services. For heavier business logic in a component, Sling Models should be used. The next exercise will cover Sling Models.

1. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the /apps/training/components/structure/header node.
3. Right-click the header node, and click Create > Create File. The Create File dialog box opens.
4. Enter header.js in the Name field and click OK. The file is created.

To add code to the header.js file:

5. Double-click the **header.js** file to open the script for editing.
6. Use the code from **header.js** in the exercise folder provided to you.
7. Click **Save All**.
8. Review the code you added now. In the following code snippet:

   a. Line 4 helps the language root of the website to display the child pages in navigation.

   b. Line 12 helps log a message in the server-side Javascript.

   c. Line 14 helps filter the child pages only if **Hide in navigation** page property is selected.



```
1  "use strict";
2  use(function() {
3      var items = [];
4      var root = currentPage.getAbsoluteParent(2);
5
6      //make sure that we always have a valid set of returned items
7      //if navigation root is null, use the currentPage as the navigation root
8      if(root == null){
9          root = currentPage;
10     }
11
12     log.info("################NavRoot Page: " + root.title);
13
14     var it = root.listChildren(new Packages.com.day.cq.wcm.api.PageFilter());
15     while (it.hasNext()) {
16         var page = it.next();
17         items.push(page);
18     }
19
20     var anon = (request.getRequestParameter("anonymous") != null) ?
21             request.getRequestParameter("anonymous").toString() : "false";
22
23     return {
24         items: items,
25         rootPage: root,
26         anonymous: anon
27     }
28 });
```

To update the code in the header.html page:

9. Double-click the **header.html** to open the script for editing.
10. Update the **header.html** script by replacing the existing code with new code from **js_header.html** in the exercise folder.
11. Click Save All.

12. Open **header.html** to view the code you added. Notice the following:

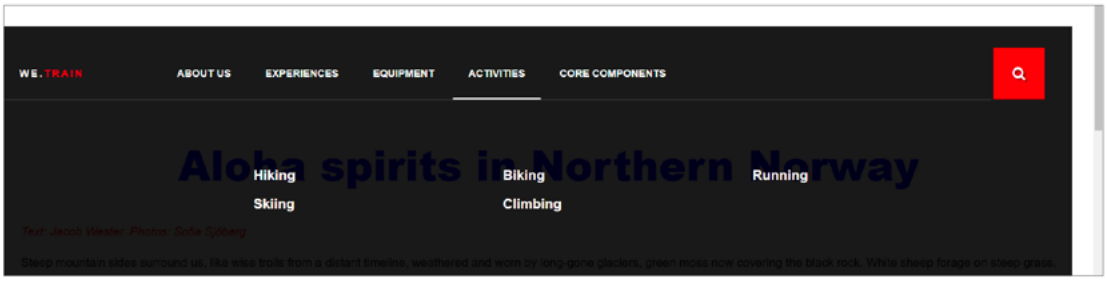
    a. Line 2: data-sly-use.header="header.js" helps define the header variable with a HTL Javascript Use-script.

    b. **${header.rootPage.path}** and **${!header.anonymous}** are now able to render based on the JavaScript.

    c. Line 53: data-sly-repeat="${header.items}". Instead of using the child pages of the current page, the HTL statement requests the items object from the header.js script.

To view the changes on the English page:

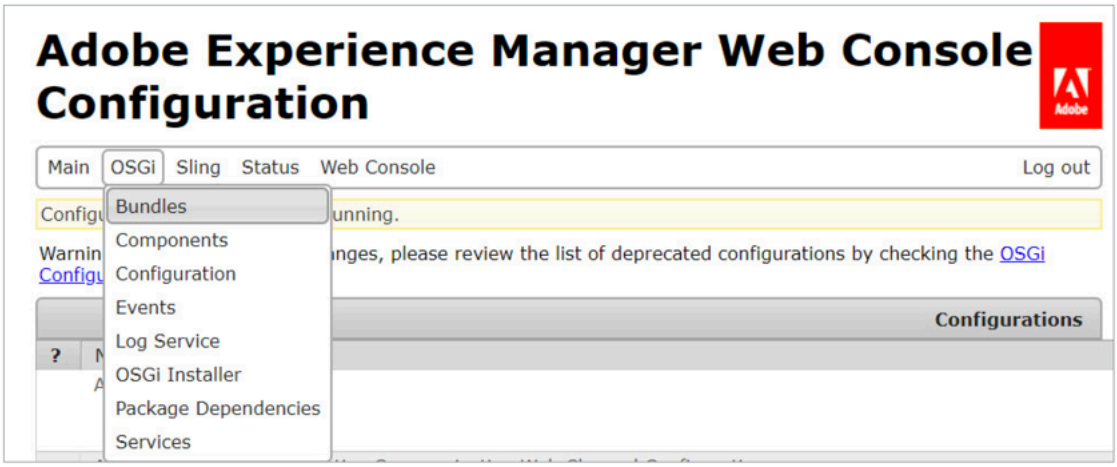
13. Navigate to the English page of the We.Train site, and refresh the page. If the English page is not open, click http://localhost:4502/editor.html/content/we-train/en.html The English page opens.
14. Click **Preview** from the page toolbar, and hover the cursor over the Activities subpage, as shown. The header component has the same functionality as in the previous exercise.



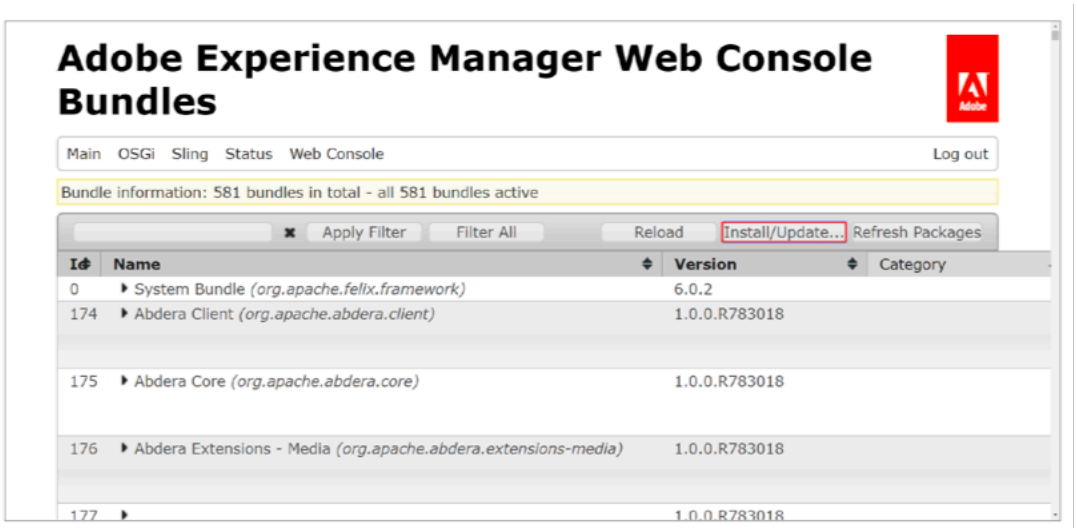
Exercise 3: Add a Sling Model business logic to the header component

In this exercise, you will use a Sling Model for the business logic of the component. Using Sling Models enable heavier computation as well as extensibility for headless implementations. For convenience, you will upload the **we.train.core-0.0.1-SNAPSHOT.jar** file, which contains a Sling Model. This Sling Model implements the exact same business logic as the JavaScript file you created earlier.
To install the bundle:

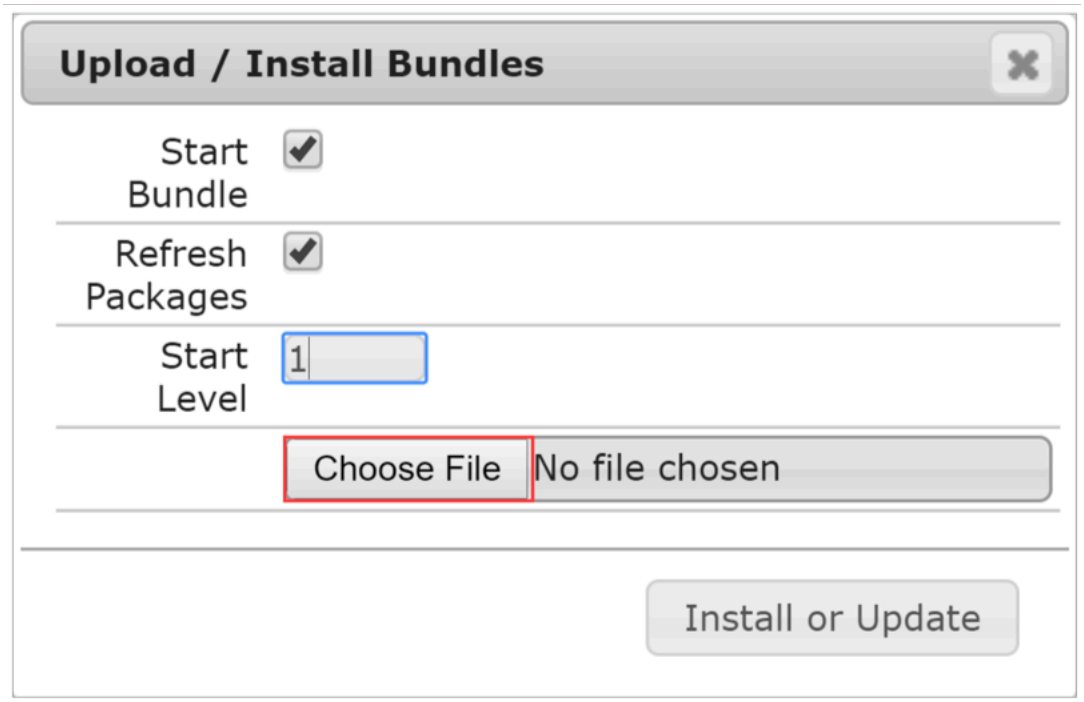
1. In your AEM author instance, **click Adobe Experience Manager** from the header bar, and click the Tools icon. The Tools console opens.
2. Click **Operations** > **Web Console**. The Adobe Experience Manager Web Console Configuration page opens.
3. Click OSGi from the header bar, and click Bundles from the drop-down menu, as shown. The Adobe Experience Manager **Web Console Bundles page** opens.



4. Click **Install/Update** from the actions bar, as shown. The **Upload/Install** Bundles dialog box opens.



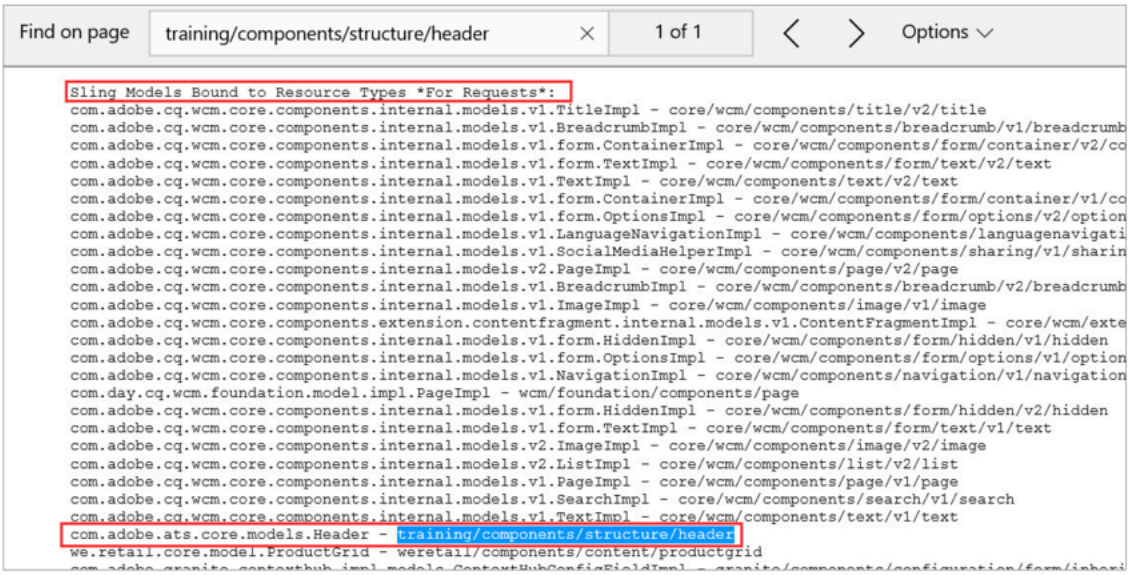
5. Select the **Start Bundle** and Refresh Packages checkboxes, enter 1 in the **Start Level** field, and click **Choose File**, as shown. The **Open** dialog box appears.



6. Navigate to the exercise folder on your file system and double-click the **we.train.core-0.0.1-SNAPSHOT.jar** file. The file is selected.
7. Click **Install or Update** in the Upload / Install Bundles dialog box. The Adobe Experience Manager **Web Console Bundles** page refreshes when the installation is complete.

To verify if the bundle is installed successfully on your instance:

8. Click **Status** on the header bar and select **Sling Models** from the drop-down menu. The Adobe Experience Manager Web Console Sling Models page opens.
9. Press Ctrl + F (Windows) or Command + F (Mac) on your keyboard and type **training/components/structure/header** in the search field. Notice that the com.adobe.ats.core.models.Header - training/components/structure/header is under **Sling Models Bound to Resource Types** *For Requests (if you cannot find this, use Ctrl+F or Command F). This confirms the bundle is installed successfully.

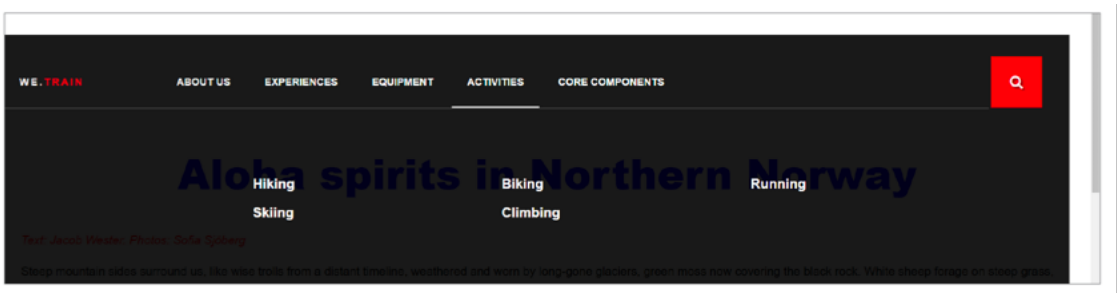


To update the code on the header.html page:

10. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
11. Navigate to the /apps/training/components/structure/header node.
12. Double-click the header.html to open the script for editing.
13. Update the **header.html** script by replacing the existing code with new code from **model_header.html** file provided to you.
14. Click Save All.
15. Review the code you added in **header.html**. Notice that the model_header.html script is the same as js_header.html, except you are requesting the Sling Model rather than the Javacript.

To view the changes on the English page:

16. Navigate to the **English** page of the **We.Train site**, and refresh the page. (If the English page is not open, click http://localhost:4502/editor.html/content/we-train/en.html).
17. Click **Preview** from the page toolbar, and ensure you can see the subpages of the Activities page, as shown:



Configuring the Edit Dialog

Dialogs are a key element of a component as they provide an interface for authors to configure and provide input to the component.

Dialogs are a part of the Granite UI Foundation that uses Coral 3 UI components and styles. As the dialogs you will create are an extension of the Granite UI foundation, you can use many of the properties, concepts, and design patterns defined in the Granite UI documentation.

Due to the dialog node structure complexity, you should reuse dialog node structures from other components as much as possible. Inheriting a dialog box from a core component or simply copying one are both viable options. If your component is extending a core component, it can inherit the dialogs' functionality and potentially extend it. Otherwise, if you are creating a completely unique component, study the dialogs of the core components to identify potential dialog fields you could use, copy the node structure from the core component dialog, and customize as needed.

Exercise 4: Create an edit dialog for the component

In this exercise, you will create a new component that will include a custom dialog for authoring input.

To create an edit dialog:

1. In the CRXDE Lite page (http://localhost:4502/crx/de.), navigate to the /apps/training/components/structure folder.
2. Right-click the structure folder and click Create > Create Component. The Create Component dialog box opens.
3. Enter the following values in the dialog box:

   a. Label: pagehero

   b. Title: Page Hero

   c. Group: We.Train.Structure
4. Click Next.
5. Leave the Advanced Component Settings as is and click OK. The component is created.
6. Click Save All from the actions bar.
7. Expand the pagehero node in CRXDE Lite and double-click pagehero.jsp. The editor opens on the right.
8. Delete the existing content and click Save All.
9. Right-click **pagehero.jsp**, select Rename, and change the file name to pagehero.html. Click Save All.

To add code to the pagehero.html page:

10. Double-click the **pagehero.html** to open the script for editing.
11. Update the **pagehero.html** script by replacing the existing content with new code from **purehtl_pagehero.html**.

Note: purehtl_pagehero.html has business logic directly in the HTL. Using this method, a front-end developer can create the entire component. Sometimes, content may need to be rendered outside of an AEM site. Using a Sling Model with your component can easily accomplish this. If you like to see the **pagehero** component with a Sling model, use model_pagehero.html rather than purehtl_pagehero.html. You need to also ensure **we.train.core-0.0.1-SNAPSHOT.jar** is installed as it contains the **Hero.java** model.

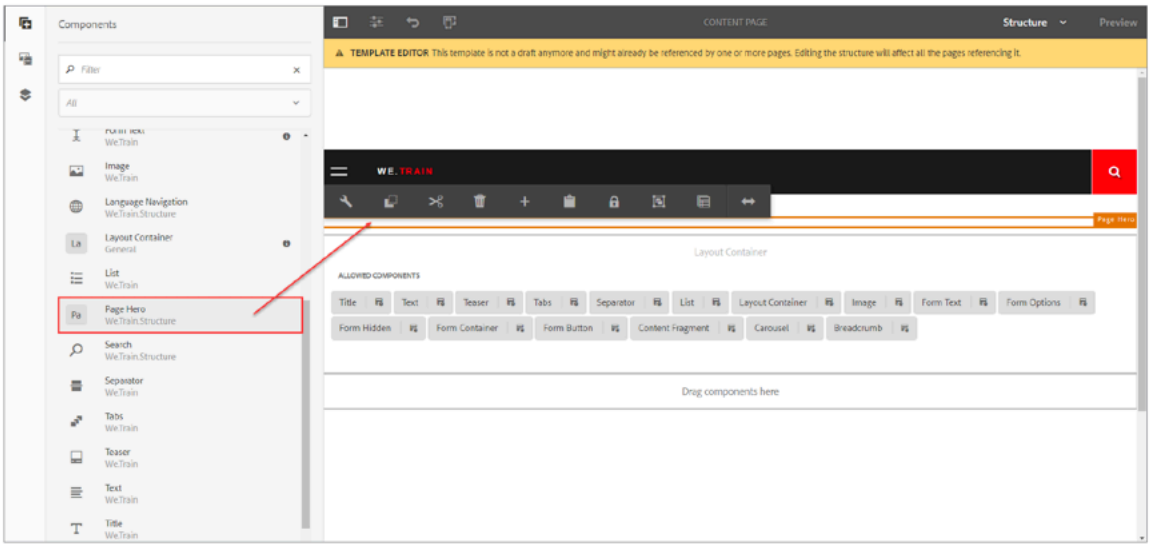
12. Review the code you just included in pagehero.html. Notice that the HTL script uses the global variables properties and resource, which helps return values persisted by the dialog box. As you do not have a dialog box yet, the values will not return anything.
13. Click Save All.
14. Before, you add the pagehero component to the template, you need to add a **cq:dialog** node to the **pagehero** component.
15. Select the pagehero component and click **Create** > **Create Node**. The Create Node dialog box opens.
16. Update the following values:

    a. **Name: cq:dialog**

    b. Type: nt:unstructured
17. Click OK and save the changes.
18. Click CRXDE Lite from the header bar to navigate back to AEM.

To add the **pagehero** component to the template:

19. **Click Tools** > **Templates**. The Template console opens.
20. Click the We.Train folder to open it, and select the Content Page template by clicking the Select icon (checkmark).
21. Click Edit (e) from the actions bar. The Template Editor opens.
22. Click the Toggle Side Panel icon, if not already selected, and click the Components icon from the left panel. Notice that the Page Hero component is available in the Components browser.
23. Drag the Page Hero Component from the left panel above the Layout Container below the Header component, as shown. The Page Hero component is added to the template.

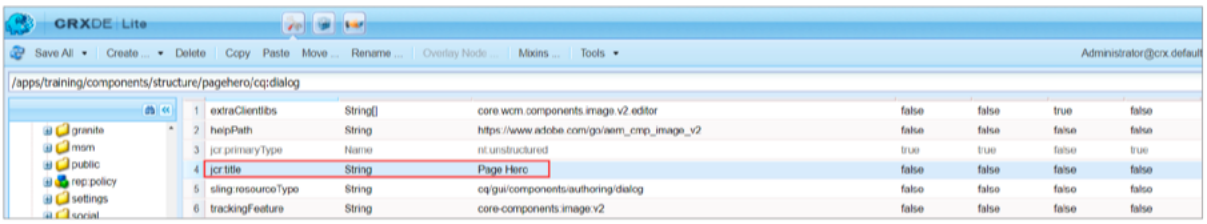


Now that you have added the Page Hero component to the template, you can start to build and test the dialog box.

24. Go to http://localhost:4502/crx/de. The CRXDE Lite page opens.

In AEM, it is a best practice to reuse a node structure rather than recreating it. You can copy the Core Image dialog box, and then modify it according to your needs.

25. Navigate to the **/apps/training/components/structure/pagehero** node.
26. Delete the cq:dialog node below the **pagehero** node.
27. Click Save All.
28. Navigate to the /apps/core/wcm/components/image/v2/image node.
29. Right-click the cq:dialog node, and click Copy. The node is copied.
30. Navigate to the /apps/training/components/structure folder.
31. **Right-click the pagehero component and click Paste. The cq:dialog node of Core Image is added to the pagehero component.**
32. Click Save All.
33. Select the cq:dialog node, and on the Properties tab, double-click **jcr:title**, and update the value as Page Hero, as shown:



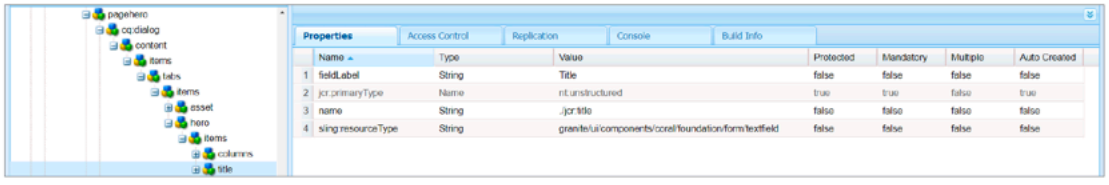
34. Right-click the **extraClientlibs** property and click Delete to delete it.

35. Similarly, delete the **helpPath** property.
36. 36. Click Save All.
37. Navigate to the **/apps/training/components/structure/pagehero/cq:dialog/content/items/tabs/items/metadata/items/columns** node.
38. Right-click the columns node and click Delete to delete it.
39. Click Save All.
40. Navigate to the **/apps/training/components/structure/pagehero/cq:dialog/content/items/tabs/items/metadata** node.
41. Right-click the metadata node and click Rename to rename it.
42. Rename it as **hero**.
43. Click Save All.
44. Select the hero node, and in the Properties tab, double-click **jcr:title**.
45. In the **Value** field, enter **Hero**.
46. Click Save All.
47. Right-click the items node that is below the hero node and click **Create > Create Node**. The Create Node dialog box opens.
48. Update the following details:

    a. Name: title

    b. Type: nt:unstructured
49. Click OK > Save All to save the changes.
50. Select the title node, and add the following properties, as shown:

Name	Type	Value
fieldLabel	String	Title
name	String	./jcr:title
sling:resourceType	String	granite/ui/components/coral/foundation/form/textfield



51. Click **Save All**.

Similar to the title node, you will create **heading, buttonLabel**, and buttonLink form fields below the **/hero/items** node. Instead of creating the formfields, you will copy/paste the title node and update the required values accordingly.

52. Right-click the title node and click Copy to copy it.
53. Ensure you have selected the /hero/items node, right-click items, and click Paste. The Copy of title node is created.
54. Rename the Copy of **title node to heading**.
55. Select the heading node and update the values of the following properties:

Name	Type	Value
fieldLabel	String	Heading
name	String	./heading
sling:resourceType	String	granite/ui/components/coral/foundation/form/textfield

56. Click Save All.
57. Perform steps 51 and 53 and create **buttonLabel** and **buttonLink** nodes below the /hero/items node.
58. Select the **buttonLabel** node and update the values of the following properties:

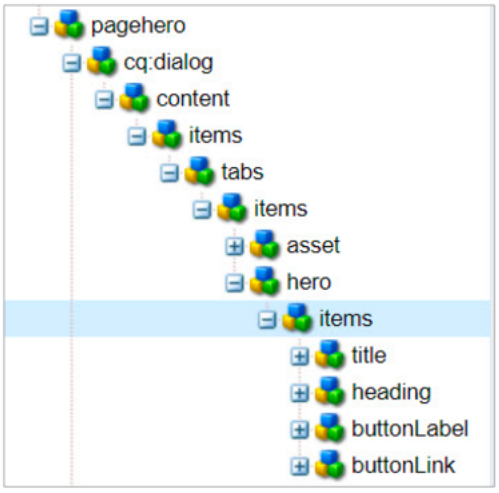
Name	Type	Value
fieldLabel	String	Button Label
name	String	./buttonLabel
sling:resourceType	String	granite/ui/components/coral/foundation/form/textfield

59. Click Save All.
60. Select the **buttonLink** node and update the values of the following properties:

Name	Type	Value
fieldLabel	String	Button Link
name	String	./buttonLinkTo
rootPath	String	/content/we-train
sling:resourceType	String	granite/ui/components/coral/foundation/form/pathbrowser

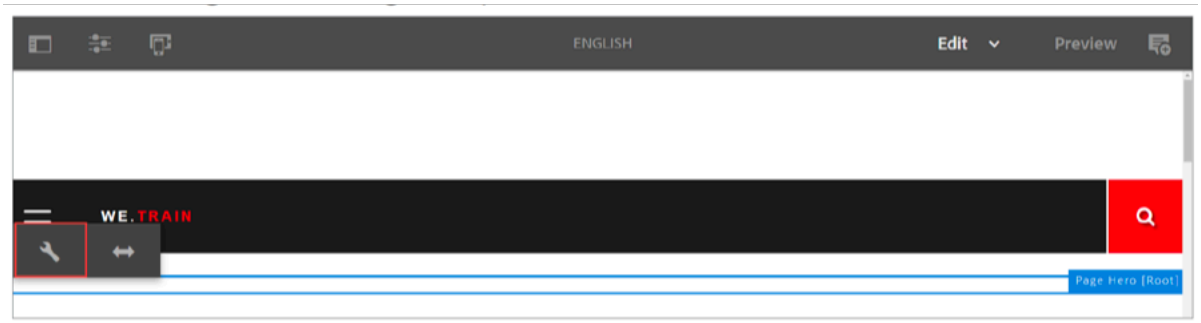
61. Click **Save All**.

Your form fields structure should look similar to the one given in the below screenshot:

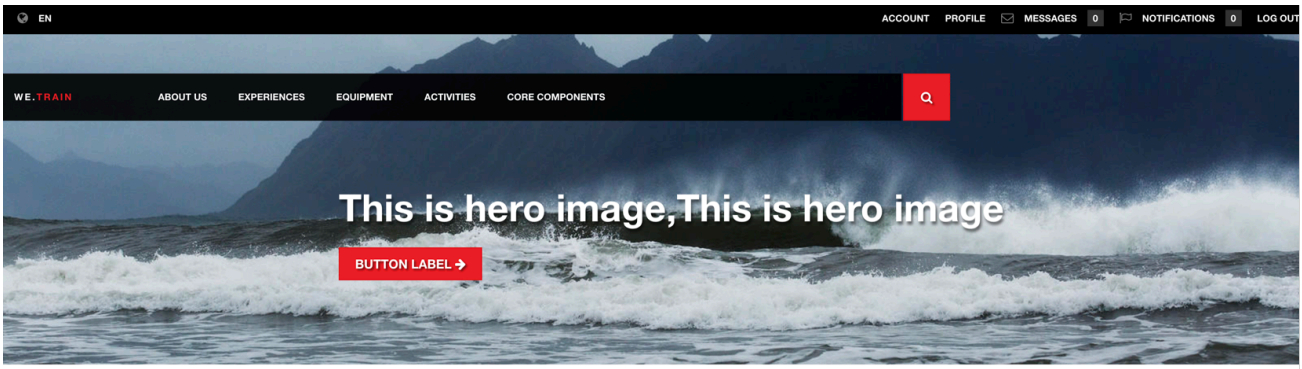


To enable the dialog box on the page, you must unlock the Page Hero component in the template.

62. Navigate back to the Content Page template. If it is not open, click CRXDE Lite from the header bar, and click Tools > Templates. The Templates console opens.
63. Navigate to the **We.Train** folder, select the Content Page template, and click Edit (e) from the actions bar. The **Template Editor** opens.
64. Select the **Page Hero** component, at the top of the page, and click the **Unlock structure component** icon from the component toolbar. The **Unlock** dialog box opens.
65. Click **Unlock.**
66. Now that the Hero component is added to the template, you can test the component on a page. Navigate to the English page of the We.Train site and edit it. Observe that the Hero component is now available on the page. If you hover the cursor over the component, a blue box appears, indicating that the component is editable.
67. Hover over the **Page Hero[Root]** component and click the **Configure** (wrench icon) icon, as shown. The Page Hero dialog box opens.



68. On the **Asset** tab, drop an asset or browse for a file to upload an image.
69. Click the **Hero** tab and add a title in the **Title** field.
70. Click **Done** (checkmark) to save the changes. If you see an image with a title, you have successfully created a dialog box with your component.



Exercise 5: Add an editconfig node to the component

In this exercise, you will add an cq:editConfig node to the Page Hero component. Instead of recreating an existing node structure, you will copy the Core Image component, add it to the Page Hero component, and update the required values accordingly.

To add an **editConfig** node:

1. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the /apps/core/wcm/components/image/v2/image node.
3. **Right-click** the **cq:editConfig** node, and click Copy. The node is copied.
4. Navigate to the /apps/training/components/structure folder.
5. Right-click the **pagehero** component and click Paste to paste the node you copied in step 3. The **cq:editConfig** node is added.
6. Right-click the **cq:inplaceEditing** node, which is below the **cq:editConfig** node, and click Delete.
7. Click Save All.
8. Navigate to the **/apps/training/components/structure/pagehero/cq:editConfig/cq:dropTargets/image** node, select the parameters node, and add the following **property**:

Name	Type	Value
sling:resourceType	String	training/components/structure/pagehero

9. Click **Save All**.

To test if the **cq:editConfig** is added to the Page Hero component:

10. Navigate to the English page of the **We.Train** site. (http://localhost:4502/editor.html/content/we-train/en.html
11. Click the **Toggle Side Panel** icon from the page toolbar and click the Assets icon. The Assets browser opens.

12. Drag an image from the **Assets** browser onto the Page Hero component. The image is added to the component. This indicates the success of this exercise. You used cq:editConfig to add the ability of your Page Hero component to accept content through the drag-and-drop feature.

Exercise 6: Add a component client library to the component

Now that you have configured the hero component as per your needs, you want to apply a design so that the hero image in the header looks more appealing. In this exercise, you will create a component client library that will have a specific design for the hero component.

To add a component clientlib:

1. Click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/component/structure** folder.
3. Right-click the **pagehero** component and click **Create** > **Create Node**. The Create Node dialog box opens.
4. Provide the following details:

   a. **Name: clientlibs**

   b. **Type: cq:ClientLibraryFolder**
5. Click OK > Save All.
6. Select the **clientlibs** node and add the following properties:

Name	Type	Value
allowProxy	Boolean	true
categories	String[]	we.train.pagehero

7. Click **Save All**.
8. Right-click the **clientlibs** folder, and click **Create** > **Create Folder**. The **Create Folder** dialog box opens.
9. Enter **css** in the Name field, and click OK.
10. Click **Save All**.
11. Right-click the css folder and click **Create** > **Create File**. The Create File dialog box opens.
12. Enter **pagehero.css** in the **Name** field and click OK.
13. Click **Save All**.

To add code to the pagehero.css file:

14. Double-click **pagehero.css** to open the script for editing.
15. Navigate to the exercise file provided to you, and use the code in **pagehero.css** file to update the pagehero.css script in CRXDE Lite.
16. Click Save All.

17. Right-click the **clientlibs** folder, and click **Create** > **Create File**. The Create File dialog box opens.
18. Enter **css.txt** in the Name field, and click OK.
19. Click **Save All**.
20. Double-click css.txt to open the file for editing.
21. Add **css/pagehero.css** to the **css.txt** file.

To include the component client library at the top of the page, you need to embed it to the clientlibs-base.

22. Navigate to the **/apps/training/clientlibs** folder, and select **clientlib-base** folder.
23. On the **Properties** tab, double-click the embed property to edit its value. The **Edit embed** dialog box opens.
24. Click +, **add we.train.pagehero** in the field, and click **OK**. This will add the css for the pagehero component at the top of the page.
25. Click **Save All**.

To observe the changes from the clientlib on the Page Hero component:

26. Navigate to the English page of the We.Train site.

    (http://localhost:4502/editor.html/content/we-train/en.html)
27. Click the **Page Information** icon from the page toolbar, and click **View as Published** from the drop-down menu. The page opens in a new tab.
28. Right-click on the page, and click **View page source** from the list. The source code opens in a new tab of the browser.
29. Click the clientlib-base.css line, as shown.



30. Search for **pagehero** in the CSS file. The client library should now include the CSS from the pagehero component. You will also see the we-heroImage class names relating to the Hero component. You have successfully added CSS to your component using a client library and verified that it is delivered along with the page.

```
.pagehero{
        position: relative;
    top: -170px;
    margin-bottom: -170px;
    overflow: visible;
}
.we-HeroImage-title,
.we-HeroImage p {
  font-family: inherit;
  font-weight: 700;
  line-height: 1.1;
  color: inherit;
  margin-top: 24px;
  margin-bottom: 12px;

}
.we-HeroImage-title {
  font-size: 48px;
  font-weight: 900;
  line-height: 1.29166667;
  display: block;
  margin: 0 auto 20px;
  text-shadow: 2px 4px 3px rgba(0, 0, 0, 0.5);
}
.we-HeroImage p {
  font-size: 24px;
  font-weight: 400;
  /* line-height: 2.58333333; */
  margin: 0 auto;
}
.we-HeroImage {
/*       */
  margin-bottom: 40px;
  color: #ffffff;
  background-color: #eeeeee;
  min-height: 460px;
  padding: 40px 0;
  background: #555555 50% no-repeat;
  background-size: cover;
  position: relative;
  text-align: center;
```

Design Dialogs

Design dialogs are similar to the dialogs used to edit and configure content, but they provide the interface for authors to configure and provide the design details for the component.
The **cq:design** _ dialog (**nt:unstructured**) node type is used to create a design dialog.

Design Configuration through Content Policies

In the template editor, policies are used to configure component design. For example, policies specify a component's design configurations, allowed components for a container, and map asset to components.

Configuring a template editor's policy is similar to a Static template's design dialog. To define and access a new policy:

1. Create a policy configuration dialog: You can define the component's policy dialog by adding a cq:design_dialog to the component.
2. Configure the policy: After adding the design dialog, when you open the template in Structure mode, and click the component, the Policy icon will be available to configure the design properties in the component toolbar.
3. Access the policy: You can access the component's policy configuration through ContentPolicyManager.

Policy Storage

Policies are stored in the following location by default:
/conf/project_name/settings/wcm/policies/component_name/policy_randomNumber

You can also find policies in the /apps or /libs folder. In this case, the resource will be resolved in the following order**: /conf, /apps, and /libs**.

The policies are stored centrally for a project. This gives the authors the ability to share a design policy among multiple templates. Template-policy mapping is done by referring policy through:
cq:policy=/conf/project_name/settings/wcm/policies/component_name/policy_randomNumber

Exercise 7: Create a design dialog for a component

In this exercise, you will create a new component with a design dialog, and then create a supporting policy that will be applied to all pages created from the template.

1. Ensure the CRXDE Lite tab is open, or click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/components** folder.
3. Right-click the structure folder and click **Create** > **Create Component**. The Create Component dialog box opens.
4. Provide the following details:

   a. Label: footer

   b. Title: We.Train Footer

   c. Group: We.Train.Structure
5. Click Next and click OK. The component is created.
6. Click **Save All**.
7. Expand the footer component and double-click footer.jsp. The editor opens on the right.
8. Delete the existing content and click Save All.
9. Right-click footer.jsp, select Rename, and change the name to **footer.html.** Click Save All from the actions bar.

10. Click save all from action bar.

To add code to the footer.html page:

11. Double-click the footer.html to open the script for editing.
12. Replace the existing content with code from the model_footer.html exercise file provided to you. The footer component uses a Sling Model called Footer.java. The bundle, we.train.core-0.0.1-SNAPSHOT.jar you installed previously already has this model built. If the bundle is not installed, you can install it through the Web Console using the **we.train.core-0.0.1-SNAPSHOT.jar** provided to you.

You must add a cq:dialog node to add the footer component to the template:

13. Select the footer component, and click **Create** > **Create Node**. The Create Node dialog box opens.
14. Provide the following details:

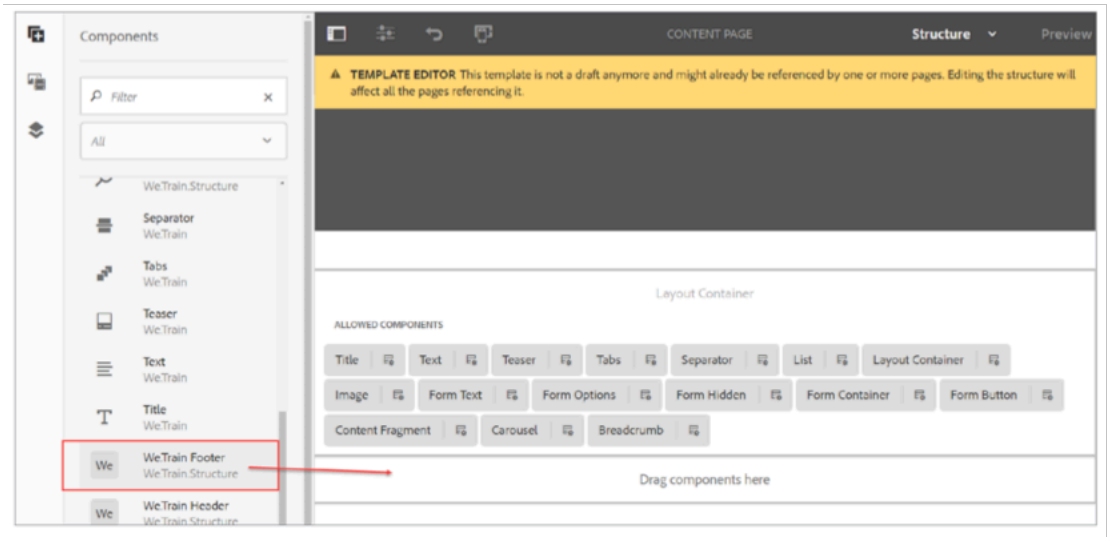
    a. Name: cq:dialog

    b. Type: nt:unstructured

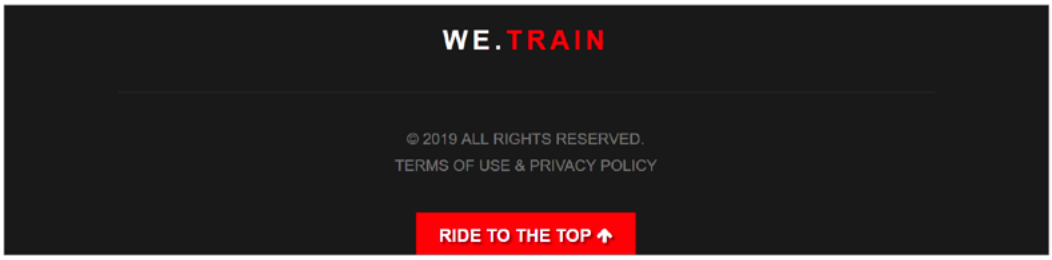
15. Click OK and click **Save All**.

To add the footer component to the template:

16. In your AEM author instance, click **Adobe Experience Manager** from the header bar, click the **Tools** icon, and click **Templates**. The **Templates console** opens.
17. Click the **We.Train** folder, select the **Content Page** template, and click Edit (e) from the actions bar.
18. Click the **Toggle Side Panel** icon from the page toolbar, and click the Components icon. The Components browser opens. Notice that the We.Train Footer is available.
19. Drag the **We.Train** Footer component from the Components browser onto the **Drag components here** placeholder, as shown. The component is added.



20. Refresh the English page and notice how the footer is rendering.



Sometimes, organizations may want to update and change the links on the footers without involving the development team. This can be achieved by creating a custom content policy for the footer.

To create a content policy, you will build a design dialog that helps the template authors specify the links for the footer. Rather than building the design dialog from the beginning, you will build it based on the list component dialog. To see this dialog:

21. Ensure the **English** page is open.
22. Select the **Drag components** here area, click the Insert component icon (+ icon), and add a List component.
23. Select the **List** component, and click the Configure (wrench) icon from the component toolbar. The List dialog box opens. Observe the dialog box and the options it provides.
24. Click X (Cancel) to close the dialog box.

To create a design dialog:

25. In CRXDE Lite, navigate to the **/apps/training/components/structure** folder.
26. Right-click the footer node, and click **Create** > **Create Node** The Create Node dialog box opens.
27. Provide the following details:

    a. Name: cq:design_dialog

    b. Type: nt:unstructured.
28. Click OK. The node is created.
29. Click Save All.
30. Select the **cq:design_dialog** node and add the following properties:

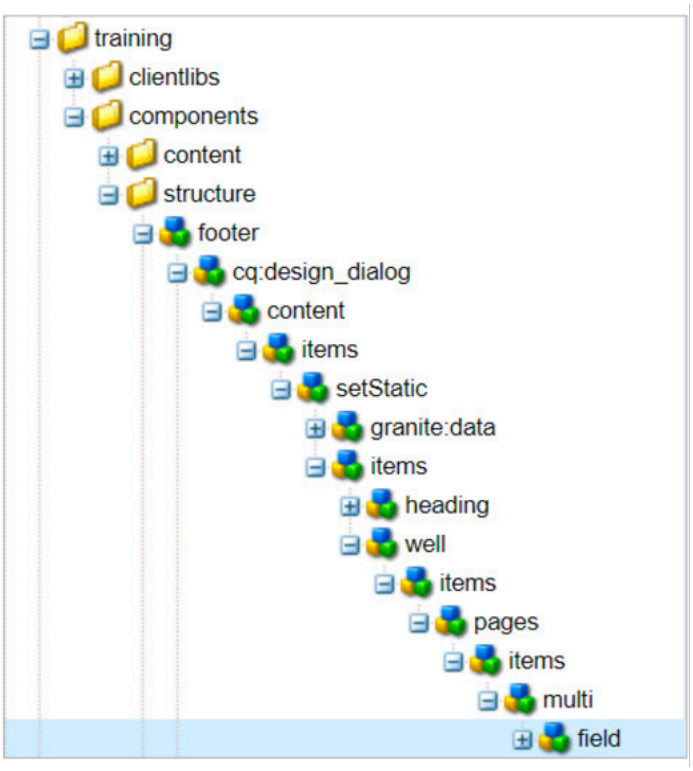
Name	Type	Value
sling:resourceType	String	cq/gui/components/authoring/dialog
jcr:title	String	Footer Policy

31. Click Save All.
32. Navigate to the **/apps/core/wcm/components/list/v2/list/cq:dialog** node, select the content node, and click Copy from the actions bar.
33. Navigate to the **/apps/training/components/structure/footer** node, select the **cq:design_dialog** node, and click Paste.

34. Click Save All.

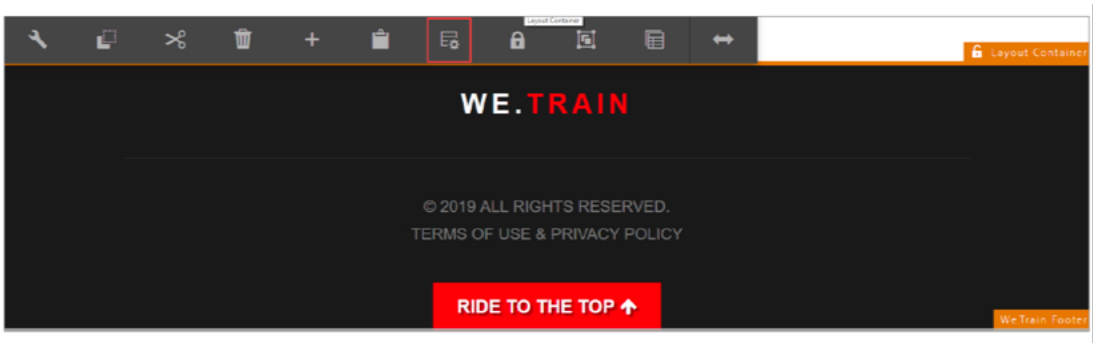
To remove all the extra fields from the dialog and customize it to your footer component.

35. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items/tabs/items** node, right-click the **itemSettings** node, and click Delete. The node is deleted.
36. Click Save All.
37. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items/tabs/items/listSettings/items/columns/items/column/items** node.
38. Right-click the **setStatic** node, and click Copy.
39. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items** node and paste the node you copied in the previous step.
40. Click Save All.
41. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items/setStatic** node.
42. On the Properties tab, right-click the **granite:class** property, and click Delete. The property is deleted from the node.
43. Click Save All.
44. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items** node.
45. Right-click the tabs node, and click Delete.
46. Navigate to the **/apps/training/components/structure/footer/cq:design_dialog/content/items/setStatic/items/well/items/pages/items/multi/field** node. Notice that the property name, ./pages holds the fixed list. Your folder structure should look similar to the one shown in the below screenshot:



Now that the design dialog is built, test it on the page.

47. Open the Content Page template, select the Footer component, and click the Policy icon from the component toolbar, as shown. The Footer Policy dialog box opens.



48. In the Policy section, in the Policy Title field, type **We.Train Footer Policy**.
49. In the Properties section, under **Options for Fixed List**, click Add, and type **/content/we-retail/us/en/experience**.
50. Click Done (checkmark icon).
51. Navigate to the **English** page of the **We.Train** site. Notice that the footer is updated with a link to the Experience page that you added in the previous step.
52. Click **Preview** from the page toolbar, and then click **Experience** in the footer to navigate to the **Experiences** page of the We.Retail site.

53. Open other pages of the We.Train site and observe the changes in the content policy.
You successfully created a component with a design dialog to allow new options to be set in the component's content policy. You also saw that the options set in a component's content policy, through the design dialog, will affect all pages in the site using that policy.